Rapport de stage

Fred Belanger
felix.belanger@universite-paris-saclay.fr

16 Avril 2026

Abstract

On a fait des bêtises sur les problèmes de typage

Contents

1 Introduction . . . 1
2 Présentation des types ensemblistes . . . 2
2.1 Définition d'un type ensembliste . . . 2
2.2 Existence d'une forme normale disjonctive . . . 3
3 Génération d'un témoin . . . 4
3.1 Définition d'un témoin . . . 4
3.2 Définition de $w \gg t$ . . . 4
4 Algorithme . . . 5
4.1 Présentation de l'algorithme . . . 5
4.2 Preuve de terminaison de l'algorithme . . . 5
4.3 Preuve de sûreté A RETRAVAILLER CF TABLEAU . . . 6
5 Extensions . . . 6
5.1 Tag . . . 7
5.2 Records . . . 7
6 Implémentation . . . 7
6.1 Cas de bases . . . 7
6.1.1 Int . . . 7
6.1.2 Enum . . . 7
6.1.3 Arrow . . . 8
6.2 Constructeurs . . . 8
6.2.1 N-uplets . . . 8
6.2.2 Tags . . . 8
6.2.3 Records . . . 8
Bibliography . . . 9

1 Introduction

Le but du stage a été de créer une fonction permettant, pour tout type non-vide, de trouver un habitant de ce type, appelé témoin. Ce témoin permet d'afficher un exemple lors de la diffusion d'un message d'erreur pour mauvais typage.

Exemple On a une fonction $f : \text{Int} \to \text{Int}$ et une variable $x : \text{Int} \vee \text{Bool}$. Alors l'application $f(x)$ est mal typée, car $\text{Int} \vee \text{Bool}$ n'est pas un sous-type de Int. On cherche donc un témoin de $(\text{Int} \vee \text{Bool}) \setminus \text{Int} = \text{Bool}$, ce qui peut rendre True ou False. Pour des types plus complexes, on veut automatiser le processus, ce qui amène à l'algorithme "Witness" expliqué ici.

2 Présentation des types ensemblistes

2.1 Définition d'un type ensembliste

Les types utilisés dans ce rapport ont été introduits par M. Laurent, K. Nguyen et G. Castagna dans "A Gentle Introduction to Semantic Subtyping" (Castagna 2005) et "Implementing Set-Theoretic Types" (Laurent and Nguyen 2025) comme des types ensemblistes. On les définit comme :

$$t ::= b \mid \alpha \mid t \to t \mid t \times t \mid t \vee t \mid t \wedge t \mid \neg t \mid \mathbb{0} \mid \mathbb{1} \tag{1}$$

Où $b \in \mathcal{B}$ sont les cas de bases (en particuliers les constantes $c \in \mathcal{C}$), $\alpha \in \mathcal{V}$ les variables de types, $\mathbb{0}$ le type vide et $\mathbb{1}$ le supertype de tout les types. $\mathcal{B}$ est composé de Int (l'ensemble des entiers relatifs) et Enum (l'ensemble des types énumérés, par exemple bool).

Exemple : $t = (\text{Int}, (\text{True} \to \text{False})) \vee 42$ est un type correct dans notre syntaxe.

On définit $[\![t]\!]$ comme l'ensemble des valeurs contenues dans t, ce qui nous permet de définir aussi la relation de sous-typage $t_1 \leqslant t_2$ qui est vrai si et seulement si $[\![t_1]\!] \in [\![t_2]\!]$. On peut noter qu'il existe des cas, comme Bool et Int, où $\text{Bool} \not\leqslant \text{Int}$ et $\text{Int} \not\leqslant \text{Bool}$. On a aussi que $\forall t, \mathbb{0} \leqslant t \leqslant \mathbb{1}$. On peut enfin définir l'égalité sémantique $t_1 \simeq t_2$ comme étant vrai si et seulement si $t_1 \leqslant t_2$ et $t_2 \leqslant t_1$.

Comme ces types sont définis co-inductivement, ils peuvent être définis comme des arbres infinis réguliers et contractifs.

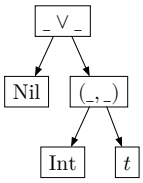
Definition 2.1

Un arbre est dit **régulier** si il a un nom fini de sous-termes possibles.

Definition 2.2

Un arbre est dit **contractif** si une branche infinie passe forcément par un constructeur (ici, $\to$ et $\times$).

Exemple $t = \text{Nil} \vee (\text{Int}, t)$ sous forme d'arbre:



On peut aussi représenter cela sous la forme d'un graphe cyclique en remplaçant la feuille t par une flèche vers _ ∨ _. Ceci nous permet d'avoir une définition de type qui n'a pas besoin de définitions explicites pour la récursion.

Enfin, on considère les types infinis comme vides, car ils sont inconstructibles en pratique (ex : $t = (\text{Int}, t) \leqslant \mathbb{0}$).

Dans un premier temps, nous allons ignorer les variables de types pour la génération du témoin.

Definition 2.3

D'après la Définition C.13 de "Polymorphic Functions with set-theoretic types" (Castagna et al. 2013), un socle $\mathfrak{I} \subset \mathcal{T}$ est un ensemble de types avec les propriétés suivantes :

- $\mathfrak{I}$ est fini,
- $\mathfrak{I}$ est clos sous les connecteurs booléens ($\vee, \wedge$ et $\neg$).
- Si $t_1 \times t_2 \in \mathfrak{I}$ ou $t_1 \to t_2 \in \mathfrak{I}$ alors $t_1 \in \mathfrak{I}$ et $t_2 \in \mathfrak{I}$.

De plus, si on a $S : \{s \mid \sqsubseteq t\}$ l'ensemble des sous-arbres de t, alors $\exists \mathfrak{I}$ tel que $S \subseteq \mathfrak{I}$.

Cette définition nous sera utile plus tard, lors de la création de l'algorithme de génération du témoin.

2.2 Existence d'une forme normale disjonctive

En pratique, un type est toujours encodé sus la forme d'une DNF (forme normale disjonctive) :

$$t ::= \bigvee_i b_i \vee \bigvee_i \left(\bigwedge_j p_1^{ij} \to p_2^{ij} \wedge \bigwedge_j \neg\left(n_1^{ij} \to n_2^{ij}\right) \right) \vee \bigvee_i \left(\bigwedge_j \left(p_1^{ij} \times p_2^{ij}\right) \wedge \bigwedge_j \neg\left(n_1^{ij} \times n_2^{ij}\right) \right) \quad (2)$$

Avec des atomes positifs, nommés p, et des atomes négatifs, nommés n. On notera que, dû à la récursivité de notre définition, on peut avoir des opérateurs d'union sous les opérateurs d'intersection (si on développe les types flèches et tuples). On appellera quand même cette forme une DNF par la suite, par mesure de simplicité. EXPLIQUER PK LES CAS DE BASES N'ONT PAS D'ATOMES NEG

On peut simplifier cette définition en transformant la partie sur les tuples en une union d'union :

Lemme 2.4

$\forall t, t \leqslant \mathbb{1} \times \mathbb{1} \Rightarrow \exists (t_1^i, t_2^i), t \simeq \bigvee_i (t_1^i, t_2^i)$

Proof: Si $t \leqslant \mathbb{1} \times \mathbb{1}$ alors :

$$\begin{aligned} t &= \bigvee\left(\bigwedge_i (p_1^i, p_2^i) \wedge \bigwedge_j \neg\left(n_1^j, n_2^j\right)\right) \\ &= \bigvee\left(\left(\bigwedge_i (p_1^i, p_2^i)\right) \setminus \bigvee_j \left(n_1^j, n_2^j\right)\right) \\ &= \bigvee\left((\wedge_i p_1^i, \wedge_i p_2^i) \setminus \bigvee_j \left(n_1^j, n_2^j\right)\right) \quad \text{par commutativité de } \wedge \end{aligned}$$

Montrons par récurrence sur $\bigvee_j \left(n_1^j, n_2^j\right)$ que t est bien une union :

On a bien $(\wedge_i p_1^i, \wedge_i p_2^i)$ qui est une union d'un seul élément.

On a maintenant $\bigvee_i (t_1^i, t_2^i)$ une union, montrons que $\bigvee_i (t_1^i, t_2^i) \setminus (n_1, n_2)$ est une union : $\bigvee_i (t_1^i, t_2^i) \setminus (n_1, n_2) = \bigvee_i ((t_1^i, t_2^i) \setminus (n_1, n_2))$

$$= \bigvee_i ((t_1^i \setminus n_1, t_2^i) \vee (t_1^i, t_2^i \setminus n_2)) \quad \text{qui est une union.}$$

Donc par récurrence, $\forall t, t \leqslant \mathbb{1} \times \mathbb{1} \Rightarrow \exists (t_1^i, t_2^i), t \simeq \bigvee_i (t_1^i, t_2^i)$. □

Ainsi, on peut réécrire la définition vu plus haut :

$$t \coloneqq \bigvee_i b_i \vee \bigvee_i \left(\bigwedge_j p_1^{ij} \to p_2^{ij} \wedge \bigwedge_j \neg\left(n_1^{ij} \to n_2^{ij}\right) \right) \vee \bigvee_i (t_1^i \times t_2^i) \tag{3}$$

3 Génération d'un témoin

3.1 Définition d'un témoin

Un témoin est un habitant de notre type, c'est à dire une valeur constante appartenant à $[\![t]\!]$. Notre but est, pour chaque cas de base, de trouver un habitant de ce cas, et pour chaque constructeur, de chercher récursivement un témoin pour chacune de ses parties.

Exemple 16 est un témoin de Int $\vee$ Bool.

Pour trouver un habitant d'une fonction f de type $t_1 \to t_2$, il faut un langage qui puisse prendre un habitant de t_1 et renvoyer le résultat de $f(\text{Witness}(t_1)) = \text{Witness}(t_2)$. On pourrait utiliser des λ termes mais cela serait complexes pour les types récursifs. Enfin, savoir que $42 \to 43$ est un habitant de Any $\to$ Int ne nous aide pas vraiment. Le procédé est donc complexe est contre-productif, nous avons donc choisis de considérer les types flèches (de la forme $\bigvee_i \left(\bigwedge_j p_1^{ij} \to p_2^{ij} \wedge \bigwedge_j \neg\left(n_1^{ij} \to n_2^{ij}\right) \right)$) comme des cas de base. On notera

$$A \coloneqq \bigwedge_i p_1^i \to p_2^i \wedge \bigwedge_j \neg\left(n_1^j \to n_2^j\right) \tag{4}$$

Cela va permettre de réécrire une nouvelle fois notre définition de type :

$$t \coloneqq \bigvee_i b_i \vee \bigvee_i A_i \vee \bigvee_i (t_1^i \times t_2^i) \tag{5}$$

Un témoin est donc soit une constante d'un cas de base, soit un type flèche, soit un tuple. On propose donc comme type témoin :

$$w \coloneqq c \mid A \mid (w, w) \tag{6}$$

Exemple : $w = (42, (\text{True}, \text{False}))$ est un témoin correct dans notre syntaxe.

3.2 Définition de $w \gg t$

On peut donc créer l'algorithme qui pour tout témoin w renvoie le type t contenant uniquement doublement (qu'on notera $w \gg t$ et appellera type_of(w)) avec les règles suivantes :

$$\frac{c \in [\![b]\!]}{c \gg b}\,\text{Base}_w \qquad \frac{w \leqslant A}{\vdash w \gg A}\,\underset{w}{\to}$$

$$\frac{\vdash w_1 \gg t_1 \qquad \vdash w_2 \gg t_2}{\vdash (w_1, w_2) \gg (t_1 \times t_2)}\,\times_w \qquad \frac{\vdash w \gg t' \qquad t' \leqslant t}{\vdash w \gg t}\,\text{Sous-type}_w$$

Exemple On veut montrer que $w = (3, (\text{True}, \text{Int} \to \text{Int}))$ est bien un témoin de $t = (\text{Int}, (\text{Bool}, \mathbb{0} \to \mathbb{1}))$:

$$\dfrac{\dfrac{3 \in [\![\text{Int}]\!]}{3 \gg \text{Int}}\,\text{Base}_w \qquad \dfrac{\dfrac{\text{True} \in [\![\text{Bool}]\!]}{\text{True} \gg \text{Bool}}\,\text{Base}_w \qquad \dfrac{\text{Int} \to \text{Int} \leqslant \mathbb{0} \to \mathbb{1}}{\vdash \text{Int} \to \text{Int} \gg \mathbb{0} \to \mathbb{1}}\,\underset{w}{\to}}{(\text{True}, \text{Int} \to \text{Int}) \gg (\text{Bool}, \mathbb{0} \to \mathbb{1})}\,\times_w}{(3, (\text{True}, \text{Int} \to \text{Int})) \gg (\text{Int}, (\text{Bool}, \mathbb{0} \to \mathbb{1}))}\,\times_w$$

4 Algorithme

4.1 Présentation de l'algorithme

On peut maintenant décrire le fonctionnement de l'algorithme qui pour chaque type t associe un témoin w (qu'on notera $t \rightsquigarrow w$ et appellera Witness(t)) qui a les règles suivantes :

$$\dfrac{w = c \leqslant b}{\Delta \vdash_s b \rightsquigarrow w}\,\text{Base}_t \qquad \dfrac{w \leqslant A}{\Delta \vdash_s A \rightsquigarrow w}\,\underset{t}{\to} \qquad \dfrac{\Delta \vdash_m t_1 \rightsquigarrow w_1 \qquad \Delta \vdash_m t_2 \rightsquigarrow w_2}{\Delta \vdash_s t_1 \times t_2 \rightsquigarrow (w_1, w_2)}\,\times_t$$

$$\dfrac{\Delta \vdash_s t_1 \rightsquigarrow w}{\Delta \vdash_s t_1 \vee t_2 \rightsquigarrow w}\,\vee_{1_t} \qquad \dfrac{\Delta \vdash_s t_2 \rightsquigarrow w}{\Delta \vdash_s t_1 \vee t_2 \rightsquigarrow w}\,\vee_{2_t} \qquad \dfrac{\Delta \cup \{t\} \vdash_s t \rightsquigarrow w}{\Delta \vdash_m t \rightsquigarrow w}\,t \notin \Delta$$

Ici, Δ est l'ensemble des types déjà visité par l'algorithme. Ainsi, si l'on retombe sur le même type, on sait que celui-ci est infini, donc vide (comme expliqué ici **RAJOUTER LIEN**). On interdit donc de repasser plusieurs par un type déjà visité avec la règle $t \notin \Delta$.

Exemple : On a $t = (\text{Int}, (\text{Int} \to \text{Bool}) \vee (\text{Bool} \to \text{Int}))$

Cela nous donne l'arbre :

$$\dfrac{\dfrac{\dfrac{w_1 = 42 \leqslant \text{Int}}{\Delta = \{\text{Int}\} \vdash_s \text{Int} \rightsquigarrow w_1}\,\text{Base}_t}{\vdash_m \text{Int} \rightsquigarrow w_1}\,\text{Int} \notin \Delta \qquad \dfrac{\dfrac{\dfrac{w_2 = \text{Int} \to \mathbb{0} \leqslant \text{Int} \to \text{Bool}}{\Delta \vdash_s \text{Int} \to \text{Bool} \rightsquigarrow w_2 = \text{Int} \to \mathbb{0}}\,\underset{t}{\to}}{\Delta = \{(\text{Int} \to \text{Bool}) \vee \text{Nil}))\} \vdash_s (\text{Int} \to \text{Bool}) \vee \text{Nil}) \rightsquigarrow w_2}\,\vee_{1_t}}{\vdash_m t = (\text{Int} \to \text{Bool}) \vee \text{Nil}) \rightsquigarrow w_2}\,t \notin \Delta}{\vdash_s (\text{Int}, (\text{Int} \to \text{Bool}) \vee \text{Nil}) \rightsquigarrow (w_1, w_2)}\,\times_t$$

On peut ainsi vérifier que $w = (42, \text{Int} \to \mathbb{0})$ est bien un témoin de $t = (\text{Int}, (\text{Int} \to \text{Bool}) \vee (\text{Bool} \to \text{Int}))$:

$$\dfrac{\dfrac{42 \in [\![\text{Int}]\!]}{42 \gg \text{Int}}\,\text{Base}_w \qquad \dfrac{\dfrac{\text{Int} \to \mathbb{0} \leqslant (\text{Int} \to \text{Bool})}{\vdash \text{Int} \to \mathbb{0} \gg (\text{Int} \to \text{Bool})}\,\underset{w}{\to} \qquad (\text{Int} \to \text{Bool}) \leqslant (\text{Int} \to \text{Bool}) \vee \text{Nil}}{\vdash \text{Int} \to \mathbb{0} \gg (\text{Int} \to \text{Bool}) \vee \text{Nil}}\,\text{Sous-type}_w}{\vdash (42, \text{Int} \to \mathbb{0}) \gg (\text{Int}, (\text{Int} \to \text{Bool}) \vee \text{Nil})}\,\times_w$$

4.2 Preuve de terminaison de l'algorithme

Théorème 4.1

Pour tout type t non-vide, $t \rightsquigarrow w$ termine.

Preuve :

On a, par définition, que $\Delta \subseteq \mathfrak{I}$, donc Δ est de taille finie. On pose u le nombre d'unions directes dans notre terme, avec $u = 1$ si l'on choisit le membre de gauche, et $u = u - 1$ si l'on choisit le membre de droite, et $s \coloneqq 1$ si $\vdash_s$, 0 si $\vdash_m$.

Alors on peut construire le nombre $(|\mathfrak{I} \setminus \Delta|, u, s)$ Montrons que, quelque que soit la règle suivie, ce nombre décroît sur l'ordre lexicographique, assurant donc la terminaison de l'algorithme :

Pour Base_t : il suffit de prendre un atome de $\bigvee_i b_i$, ce qui se fait en temps fini.

Pour $\underset{t}{\rightarrow}$: il suffit de prendre un atome de A, ce qui se fait aussi en temps fini.

Pour $\times_t$: s passe de 1 à 0 car on passe d'une règle $\vdash_s$ à des règles $\vdash_m$, on a donc bien $(|\mathfrak{I} \setminus \Delta|, u, 0) \underset{\text{lex}}{<} (|\mathfrak{I} \setminus \Delta|, u, 1)$.

Pour $\vee_{1_t}$: u passe à 1, donc on a bien $(|\mathfrak{I} \setminus \Delta|, 1, 1) \underset{\text{lex}}{<} (|\mathfrak{I} \setminus \Delta|, u, 1)$

Pour $\vee_{2_t}$: u passe à $u-1$, donc on a bien $(|\mathfrak{I} \setminus \Delta|, u-1, 1) \underset{\text{lex}}{<} (|\mathfrak{I} \setminus \Delta|, u, 1)$

Pour $t \notin \Delta$: u est susceptible d'augmenter, mais Δ augmente, donc $|\mathfrak{I} \setminus \Delta|$ décroît de 1,on a donc bien $(|\mathfrak{I} \setminus \Delta| - 1, u', 1) \underset{\text{lex}}{<} (|\mathfrak{I} \setminus \Delta|, u, 0)$

Donc par induction fondée sur l'ordre lexicographique de $(|\mathfrak{I} \setminus \Delta|, u, s)$, pour tout type $t \not\leq \mathbb{0}, t \rightsquigarrow w$ termine. □

4.3 Preuve de sûreté A RETRAVAILLER CF TABLEAU

Théorème 4.2

Pour tout type t non-vide, si $\emptyset \vdash_s t \rightsquigarrow w$ alors $w \gg t$. En d'autres termes, $\forall t, \text{type_of}(\text{Witness}(t)) \leqslant t$.

Preuve : On veut prouver $P(t) \coloneqq$ Si $\emptyset \vdash_s t \rightsquigarrow w$ alors $w \gg t$:
On prouve par induction sur les règles d'inférences de $t \rightsquigarrow w$:

Si $\vdash_s b \rightsquigarrow w$, alors par Base_t, $w = c \leqslant b$, donc $w = c \in [\![b]\!]$, donc $w \gg b$.

Si $\vdash_s A \rightsquigarrow w$, alors par $\underset{t}{\rightarrow}$, $w = w_1 \rightarrow w_2 \leqslant A$, donc par $\underset{w}{\rightarrow}$, $w \gg A$.

Supposons maintenant $P(t_1)$ et $P(t_2)$.

Montrons que $P(t_1 \times t_2)$:
Si $(t_1 \times t_2) \rightsquigarrow (w_1, w_2)$, alors par $\times_t$, $t_1 \rightsquigarrow w_1$ et $t_2 \rightsquigarrow w_2$, donc par hypothèse, $w_1 \gg t_1$ et $w_2 \gg t_2$, donc par $\times_w$, $(w_1, w_2) \gg (t_1 \times t_2)$.

Montrons que $P(t_1 \vee t_2)$:
Si $t_1 \vee t_2 \rightsquigarrow w$, alors par $\vee_{1_t}$, $t_1 \rightsquigarrow w$, donc $w \gg t_1$, or $t_1 \leqslant t_1 \vee t_2$ donc par Sous-type_w, $w \gg t_1 \vee t_2$.

Donc pour tout type t non-vide, si $\emptyset \vdash_m t \rightsquigarrow w$ alors $w \gg t$. □

5 Extensions

Des extensions ont été rajoutés aux types ensemblistes déjà présents pour couvrir certaines types précis : les n-uplets, les tags et les records. Tout ceux-ci ne sont que des versions spécifiques des tuples, ils ne seront donc pas traités sur le plan théorique. On peut donc étendre notre définition de témoin :

$$w \coloneqq i \in [\![\text{Int}]\!] \mid e \in [\![\text{Enum}]\!] \mid A \mid (w, w, ..., w) \mid \text{tag}(w) \mid \{l_i : w ... l_n : w\} \tag{7}$$

5.1 Tag

Les atomes de tags sont des tuples avec un type Tag (équivalent à un type string) en premier membre et un type t en deuxième. Ils servent à marquer certains types pour ne pas être confondus (ex : foo(Int)).

5.2 Records

Les atomes de records sont de la forme

$$r \coloneqq \{\text{bindings} : (\text{Label} \times f)\ \text{map}; \text{tail} : f; \text{exists} : (\text{Label Set} \times f)\ \text{Map}\} \tag{8}$$

- Ici, le type f est une extension de notre type t qui inclus les types optionnels (ex : $f = \text{Int?}$). On le définit comme $f \in \mathbb{1} \cup \bot$, $\bot$ dénotant le type absent.
- Le bindings est un champ où l'on place toutes les variables nommées de notre record.

Exemple $\{x : \text{Int}; y : \text{Int?}; \text{prédicat} : \text{Bool}\}$

- La tail f' indique si notre record est ouvert ou fermé (donc s'il peut avoir des variables supplémentaires non nommées).

Exemple $\{x : \text{Int};\quad y : \text{Int};;\quad \text{Int?}\}$ pour un système de coordonnées à au moins 2 dimensions.

- Pour chaque paire $(L_i : e_i)$ de exists, L_i indique les noms que la variables ne peut PAS prendre et $e_i \wedge f'$ indique son type.

Exemple $\{x : \text{Int}; y : \text{Int};; \text{Any?};; \{w; z\} : \text{Bool?}\}$ pour indiquer que les variables w et z ne peuvent pas avoir le type Bool si elles existent.

6 Implémentation

L'algorithme va essayer de trouver un témoin dans chacune des 6 grandes catégories (Int, Enum, Arrows, Tag, Tuple, Record) dans cet ordre, et s'arrêter dès qu'un est trouvé. Si aucun témoin n'est trouvé, on considère le type comme vide et on lève une exception.

6.1 Cas de bases

6.1.1 Int

Les entiers sont représentés comme des intervalles. Il existe 3 cas possibles :

- $t =]-\infty; +\infty[$: l'algorithme retourne 42.
- $t =]-\infty; i]$ ou $t = [i; +\infty[$: retourne i.
- $t = [i_1; i_2]$: retourne i_1.

6.1.2 Enum

Enum contient tout les types énumérés, incluant notamment les booléens (et les Strings ?). Il peuvent être définis de 2 manières :

- comme une union des constantes appartenant à t : on renvoie la première
- comme une union des constantes appartenant à $\neg t$: On retourne une constante de Enum de taille n, n n'étant la taille d'aucune des constantes de $\neg t$

Exemples

1. True | False donnera True.
2. Enum $\setminus$ ($a \vee$ ba $\vee$ toto) donnera abc.

6.1.3 Arrow

Comme dit plus haut, on considère les atomes de la forme $\bigwedge(t_1 \to t_2) \wedge \bigwedge \neg(t_1 \to t_2)$ comme des cas de base. On va donc prendre tout les atomes positifs de t, et y rajouter un par un les atomes négatifs de t. Si au moment d'un ajout, l'intersection des atomes positifs et négatifs donne un sous-type de t, c'est notre témoin.

6.2 Constructeurs

6.2.1 N-uplets

Les n-uplets peuvent, comme les Enum, être défini comme les éléments présents dans t ou comme les éléments présents dans $\neg t$.

- Si on a la liste des éléments présents dans t, on choisit l'élément e de plus petite arité dont aucun élément n'est vide, et on renvoie un n-uplet de la même taille et dont chaque élément est le témoin du type à la même position dans e.
- Si on a la liste des éléments présents dans $\neg t$, on trouve le plus petit entier n tel qu'il n'y ai aucun n-uplet de cette arité dans la liste, et on en renvoie un témoin (usuellement le n-uplet (0,1,...,n))

Exemples

1. $(\text{Int}, \text{Bool})$ donnera $(42, \text{True})$
2. $\text{Tuple} \setminus ((\text{Int},) \vee (\text{Int}, \text{Bool}))$ donnera $(0, 1, 2)$

6.2.2 Tags

Les Tags sont traités de la même manière que les n-uplets de taille 2, à l'exception que l'on n'opère pas de récursion sur le premier membre.

Exemples

1. foo(Int) donnera foo(42).
2. Tag \ foo(Int) donnera a(16).

6.2.3 Records

Pour générer un témoin, on va trouver un atome non vide de notre record, et opérer comme suit :

- Pour chaque paire $(l_i : f_i)$ de bindings, si f_i est requis, alors on cherche récursivement un témoin w_i de f_i et ajouter $(l_i : w_i)$ au bindings de notre témoin w.

Exemple $\{x : \text{Int}; y : \text{Int?}; \text{prédicat} : \text{Bool}\}$ donnera $\{x : 42; \text{prédicat} : \text{True}\}$.

- Si la tail f' est requise, on l'ajoute dans la tail de w.

Exemples

1. $\{x : \text{Int};; \ \ \text{Bool}\}$ donnera $\{x : 42;; \text{Bool}\}$.
2. $\{x : \text{Int};; \ \ \text{Int?}\}$ donnera $\{x : 42\}$.

- Pour chaque paire $(L_i : e_i)$ de exists, si $e_i \wedge f$ est requis, alors on crée un label $l'_i \notin L_i$, on cherche récursivement un témoin w'_i de $e_i \wedge f$ et on ajoute $(l'_i : w'_i)$ **au bindings**.

Exemple $\{x : \text{Int}; y : \text{Int} \ ;; \ \text{Any?} \ ;; \{w; z\} : \text{Bool}\}$ donnera $\{x : 42; y : 42; a : \text{True}\}$

Bibliography

[1] G. Castagna, "Semantic subtyping: challenges, perspectives, and open problems," no. 3701, pp. 1–20, 2005.

[2] M. Laurent and K. Nguyen, "Implementing Set-Theoretic Types," Nov. 2025, [Online]. Available: https://hal.science/hal-05369012

[3] G. Castagna, K. Nguyen, Z. Xu, and P. Abate, "Polymorphic Functions with Set-Theoretic Types. Part 2: Local Type Inference and Type Reconstruction," Nov. 2013, [Online]. Available: https://hal.science/hal-00880744